



A2

Grapheme-to-phoneme conversion

Abstract

This report investigates a character-level LSTM sequence-to-sequence model for grapheme-to-phoneme conversion, where input words are mapped to output phoneme sequences.

Tracey Letlape
Ltltra001@myuct.ac.za

Table of Contents

AI Usage Statement	2
OOP Design.....	2
Classes	2
Functions	2
Additional Implementation Choices:	3
DataProcessing.....	3
Training.....	3
Hyperparameter Tuning	4
Architecture Comparison	5
Evaluation.....	5
Length Analysis	6
Conclusion	6

AI Usage Statement

I used ChatGPT to help with the following:

- Clarify the assignment requirements
- Debug PyTorch dimension errors
- Improve my implementation of Seq2Seq training, greedy decoding and analysis sections.
- I reviewed, tested, and adapted the results myself, and I take responsibility for the final notebook, report and results.

OOP Design

The notebook makes use of classes and functions to allow for reusability, mainly for training and tuning as they are done for multiple models.

Classes

1. **WPDataset:**
 - Provides method to encode a word and phoneme pair into a sequence of indices, decode back, and clean tokens by removing the special token indices from the sequence of indices.
2. **LSTMCell:**
 - The class was adapted from the tutorial 3 memo provided by **Prof Francois Meyer**. Computes one LSTM time step, return new hidden state and new cell state.
3. **Encoder:**
 - Encodes the source character sequence into dense vector representation, which are processed sequentially by the LSTM cell.
 - Computes the hidden state at each time state and returns a tensor of all hidden states and tensors of the final hidden state and the cell state.
4. **Decoder:**
 - a. Decodes generates output phoneme sequences.
 - b. Depending on the context mode, the decoder either uses no additional context, encoder final hidden state for fixed context mode or computes attention-based context vectors for all encoder hidden state if context mode is attention.
5. **Seq2Seq:**
 - a. Links the encoder and decoder into one model.
 - b. The model can use teacher forcing during training where it receives previous correct target tokens and uses them to help predict the next token.
 - c. The greedy token method is defined to repeatedly predict the next token starting from <SOS> until the maximum length or <EOS> is reached. Next token is predicted as the token that maximises logits.

Functions

1. **seed_worker:** to ensure deterministic behaviour for DataLoaders.
2. **read_raw_data:** read the word and phoneme pairs from the data files.
3. **build_vocab:** build word and phoneme vocabulary from the training dataset.
4. **collate_fn:** pad sequences within a batch with zero to ensure equal lengths.

5. **train_model**: train and evaluate the model. Compute the validation and training metrics across a number of epochs.
6. **plot_results**: plots the training metrics curves.
7. **tuning**: performs a hyperparameter search given the different hyperparameter set.
8. **plot**: plots the training and validation loss curves for different models using best configurations.
9. **find_attention_wins**: using greedy decoding, find the at least five words whether one model accurately predicts the targets and the other fails.
10. **get_eval_metrics**: computes the Phoneme Error Rate (PER) and the word accuracy for a given model over a given dataset.
11. **bckt_len_accuracy**: computes the accuracy of the model predictions based on the word length.
12. **plot_len_analysis**: plot the bucket length analysis results into a bar chart, for all models on one axis.

Additional Implementation Choices:

- Used batch size = 256 to provide a reasonable number of batches for efficient training.
- Used embedding dimension = 32 and hidden size = 64 due to common usage.

DataProcessing

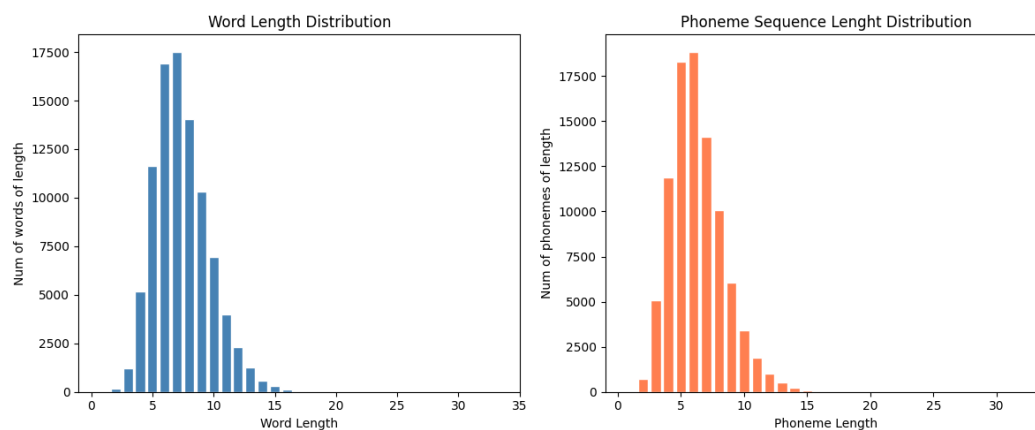


Figure 1: Word and Phoneme Distributions

- From **Figure 1** above, we can see that the training dataset is dominated by short-to-medium words.
- Longer words are mostly rare, so the models will mostly be trained and tested on words around 5-9 characters and phoneme sequences around 4-8 phonemes.

Training

To reduce overfitting, the model is trained for 10 fixed epoch, which is sufficient to visualise model behaviours. Early stopping is implemented to stop training when validation accuracy stops improving for five consecutive epochs.

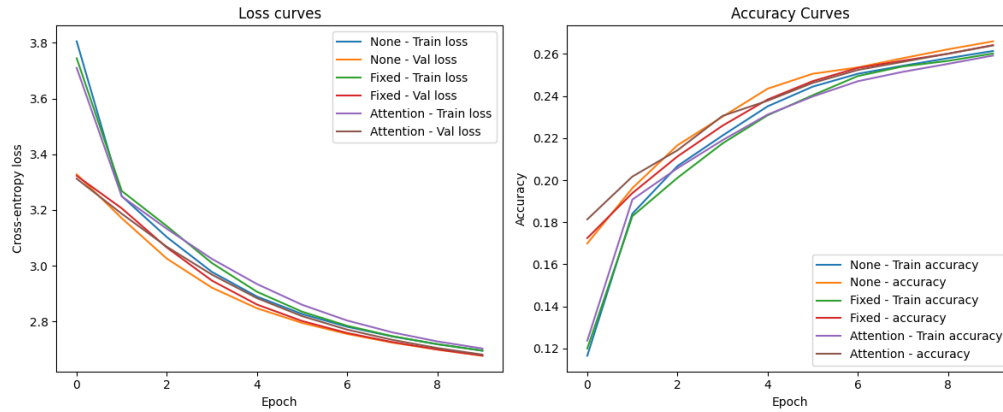


Figure 2: Baseline Training results

- Under teacher-forced training behaviour, the training and validation loss curves are steadily decreasing for all three setups, so the models are learning over the training epochs.
- The training and validation curves remain close, suggesting that there is no strong evidence of overfitting.
- The validation accuracy is still very low, around 26%, but can be better if the models trained for longer.

Hyperparameter Tuning

The learning rate (lr) in [0.001, 0.00001], embedding dimension (emb_dim) in [64, 128], and hidden size (hz) in [128, 256] were tuned with the No-Context model. Each combination of the hyperparameters was trained for 5 epochs each. The number of combinations is limited to ensure efficient training.

Tuning result are as follows:

lr	emb_dim	hz	Val_loss	Val_acc
0.001	128	256	2.146267	0.428211
0.001	64	256	2.157213	0.413362
0.001	128	128	2.271439	0.377468
0.001	64	128	2.305162	0.372799
0.00001	128	256	3.086266	0.218437
0.00001	64	256	3.153565	0.206278
0.00001	128	128	3.229106	0.199061
0.00001	64	128	3.286821	0.185884

Table 1: Tuning results

The tuning results above show that the learning rate of 0.001 performed substantially better than 0.00001. From **Figure 2**, we can see that the No-Context model results in a training accuracy of about 26% after training for 10 epochs using learning rate = 0.0001, worse than when learning rate = 0.001 and better than when learning rate = 0.00001. The table also suggests that the learning rate affects the accuracy more than the embedding dimension and the hidden size. Higher hidden size and embedding size, together with higher learning rate typically perform better.

The best configurations, lr=0.001, emb_dim=128, hz=256 were chosen to provide richer representation of tokens and maximise validation accuracy.

Architecture Comparison

Using the best configuration chosen above, three different architectures were compared: one using no context vector, one that used a fixed context vector and one that uses attention-based context vectors. **The training and validation curves are as follows:**

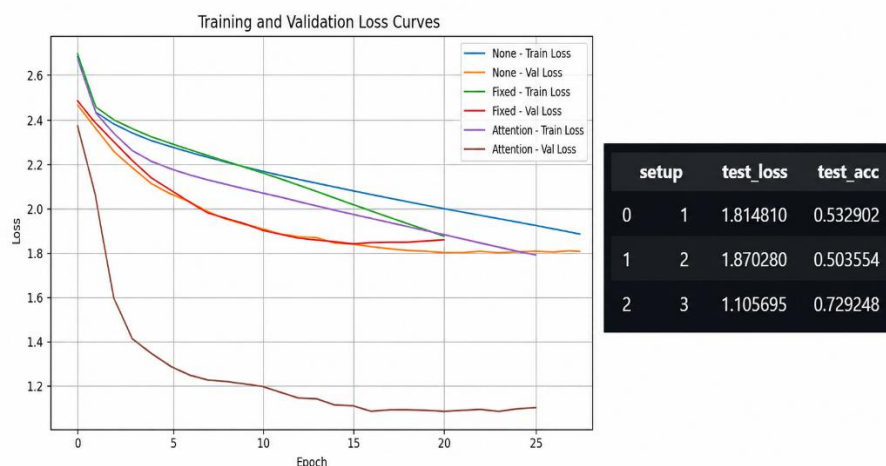


Figure 3: Architecture Comparison Results

From **Figure 3** above, we can see that attention provides the strongest decoder architecture. The fixed-context model did not improve over the no-context baseline, suggesting that a single encoder summary vector might not be sufficient for this task.

The attention model avoids this bottleneck by allowing the decoder to focus on different of the input character at each output step. The model boasts a test accuracy of around 73%.

To better understand the error pattern across models, I used at least 5 test examples to compare models. For words like “*crisper, flue*” where pronunciation depends on more than just the character, we see that the fixed context and the no fixed context models fail to map. This is because they do not focus on a different input character at each output step.

For name-like spellings words such as “*rendell, murray*”, that are context-sensitive character-to-phoneme mappings, fixed context performs worse than no context model. This is because the fixed context model compresses the entire word into one vector. If the vector is not sufficiently informative, repeatedly injecting it may confuse the decoder.

Evaluation

To better contrast the models, the Phoneme Error Rate (PER) and word accuracy were computed for each setup. **The results are as follows:**

Setup	PER	Word Accuracy
No Context	0.855321	0.013069
Fixed Context	0.914104	0.005366
Attention	0.701289	0.109832

Table 2: PER and Word Accuracy Results

Table 2 above further supports our above claim that the attention-based model provides relatively better performance than the fixed context and the no context models. However, the model does not provide absolute best performance.

This might be because words are generally not simple letter-to-sound mappings. The model may need to learn context-sensitive mappings, vowel stress, digraphs, repeated letters and weird name-like spellings.

Length Analysis

Since the Grapheme-to-Phoneme conversion assumes letter-to-sound mappings, length might also affect model performance. We run a length analysis where we contrast the different decoder designs in how they handle words of different lengths.

We compute word accuracy by word length across the three model setups. **Figure 4** below shows the results.

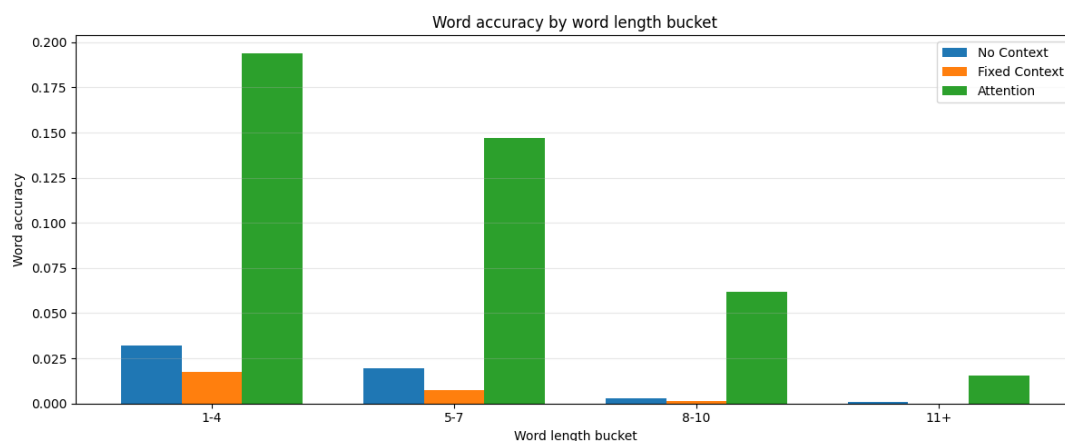


Figure 4: Length Analysis

We can see that for all three setups, increase in word length decreases word accuracy. Longer words are harder for all models as they will usually require longer phoneme sequences. Word length negatively affects performance, especially for models without attention. However, attention only mitigates this effect but does not remove it completely. Absolute word accuracy is still low though.

Conclusion

Overall, the attention-based decoder performed best among the three Seq2Seq variants tested. This suggests that, for this LSTM encoder-decoder architecture, dynamically over encoder hidden states is more effective than relying only on the decoder state or on a single fixed context vector.

However, this does not mean that the attention-based LSTM decoder is necessarily the best possible model for grapheme-to-phoneme conversion on this dataset. The absolute performance was still limited, supported by high PER and low word accuracy.

A stronger model for this type of data may be a Transformer-based encoder-decoder model, which models relationships between all input characters directly and can learn richer context-dependent spelling-to-sound mappings.